How to Defend Serverside Programming Practical Works 4–6

ORM setup: DAO, sessions, transactions · entities, Criteria, repositories · REST and transactions

Серверное программирование · variant 1 · the «ВелоГрад» bike-rental database · English edition, short

This is the short edition of the defense guide. It prepares you for the oral defense of practical works 4, 5 and 6 only. For each work you get a 30-second pitch, the Java-to-TypeScript table, a demo script, the things you must be able to explain, and likely questions with answers. Sections 4, 5 and 6 are the same as in the full guide for works 1–6.

It matches the current, minimal code. Every exercise folder contains exactly what its task sheet asks for: no comments, no tests, no extra tools, and one command per exercise (`npm start`). The reports were rebuilt from that code and from fresh runs. Every command and number in this guide was run or measured on 5 November.

I do not know exactly how your teacher runs a defense. The usual format is: the teacher opens your report, asks you to show a result, changes a parameter a little and asks why it behaves as it does. Each question is shown in English and, under it, in the Russian wording you are likely to hear; most answers have a short Russian version (“Say it in Russian”). All Russian text is in normal type, not italics.

Do not memorise word for word. Read an answer, close the guide, and say it in your own words; if you can, you understand it.

Contents

- 0. Before you go: what the tasks asked, how to run it, the one trap with work 4, checklist
- 4. Work No. 4 – ORM setup: DAO, sessions, transactions (Drizzle instead of Hibernate)
- 5. Work No. 5 – entities, Criteria API, repositories
- 6. Work No. 6 – REST service and transactions (Express instead of Spring Boot)
- 7. If something goes wrong
- 8. Cheat sheet: numbers and rules
- 9. Glossary English – Russian

0. Before you go

What the task sheets asked, and where it is

All paths are inside `C:\Users\USER\Documents\university-serverside-by-exercise`. Folder names: **pz4** = `pz4-orm-setup`, **pz5** = `pz5-entities-criteria-repositories`, **pz6** = `pz6-rest-transactions`. Each folder also has `report\` (the PDF report) and `task\` (the text of the task sheet). The code is also on GitHub: `github.com/Mohamed-Amini/university-serverside-by-exercise`.

Work	Requirement	Where it is	Status
4	Config, DAO (getSession, begin, commit), TestSrvs with 3 methods, Main	pz4\src\config.ts, DAO.ts, TestSrvs.ts, Main.ts, schema.ts	Done (Drizzle instead of Hibernate)
5	Entity for all tables; Criteria findAll; dynamic filter; CRUD; 1:N demo	pz5\src\model.ts (9 tables), relations.ts, Criteria.ts, StationDao.ts, DistrictDao.ts	Done
5	Repository; query method; @Query	pz5\src\JpaRepository.ts, StationRepository.ts, DistrictRepository.ts	Done (hand-written, see 5.4)
6	GET, POST, DELETE; JSON; DTO from several tables	pz6\src\app.ts, services.ts (RentalService → RentalDto)	Done (Express instead of Spring Boot)
6	Transactional service; rollback demo; isolation level	pz6\src\DAO.ts (transactional), services.ts (register), demo.ts; READ COMMITTED	Done
6	Demo client	pz6\public\index.html	Done

About the stack: section C of every task sheet allows an alternative stack agreed with the teacher; you told me the teacher accepted Node.js and TypeScript. Works 4–6 use Drizzle ORM instead of Hibernate and Express instead of Spring Boot on PostgreSQL 17, with the Java-to-TypeScript mapping written in each report. Nothing was uploaded to eКурсы by me; you upload the files yourself.

Old and new reports. If you already uploaded the earlier reports (the ones in `Downloads` without a folder, made before 5 November), they describe the earlier, fuller version of the code (tests, extra scripts, validation). The rebuilt reports in the exercise folders describe the current code. Make sure you defend the code that your report describes, and open the matching files.

How to run it

Works 4–6 use a separate PostgreSQL server whose data lives in the folder `.pgdata` (about 1 GB; port 54329, local only, no password); your computer's main PostgreSQL is not touched. Run in PowerShell:

```
cd C:\Users\USER\Documents\university-serverside-by-exercise\pz4-orm-setup
npm run pg:start        # start the server (works from the folder of any of works 2-6)
$env:PGCLIENTENCODING = 'UTF8'    # so Russian letters in psql are not shown as ????
& 'C:\Program Files\PostgreSQL\17\bin\psql.exe' -h 127.0.0.1 -p 54329 -U postgres -d velograd
npm run pg:stop         # when you are done
```

Work	Command (inside its folder)	What it does
4	`npm start`	prints a station name, a list of names, and adds one station (each run leaves one more station in the table).
5	`npm start`	runs every DAO and repository method and prints the results (it removes what it creates).
6	`npm start`; `npm run demo`	start: the REST server on port 8080 (open `http://localhost:8080/`); demo: runs the whole scenario in one process, including the rollback, and cleans up.

The trap: work 4 leaves a station behind

Every `npm start` in pz4 inserts one station (district 1, ‘Центральный’) and keeps it. The reports and this guide show the clean state with **300 stations**. If you ran pz4 and then pz5 or pz6, the numbers are off by one: pz5 prints `findAll: 301`, `stations 34 -> 36`, `after delete: 34`, and `GET /api/stations` returns 301 rows. Fix it with one line in psql before showing pz5 or pz6: `DELETE FROM station WHERE id > 300;`

Работа 4 каждый раз добавляет станцию. Перед показом работ 5 и 6 удалите лишние: DELETE FROM station WHERE id > 300;

- **The first start can take a while.** If the computer was switched off or crashed since the last time, PostgreSQL recovers before it accepts connections; this took about a minute once. Start the server well before the defense.
- **Do not run `npm start` in pz2 before the defense.** It drops and recreates the tables with the small data set (about 28,000 records and only 60 stations). To get the full database back (300 stations, 400,000 rentals), run `npm start` in pz3 afterwards (it detects the small data set and loads it again, about a minute).
- Only one server can use port 54329. If `pg:start` says the port is busy, the original project’s server may be running: stop it first.
- Inside psql, `\q` quits.

15-minute checklist

- Start the server and connect with psql. Run `\dt`: 9 tables. Run `SELECT count(*) FROM station;`: it must say **300**. If it says more, run `DELETE FROM station WHERE id > 300;`.
- Run `npm start` in pz4 (the output is a name, five names and `1`), clean up with the DELETE above, then `npm start` in pz5 (15 lines, `findAll: 300`) and `npm run demo` in pz6 (the whole scenario with the rollback).
- Run `npm start` in pz6 once and open `http://localhost:8080/` in the browser: the station table must appear. Stop the service with Ctrl+C.
- Open the three reports once. Know the page that shows the DAO and TestSrvs (report 4), the Criteria and the repositories (report 5), and the rollback figures (report 6).
- Read each 30-second pitch aloud once. Remember the key numbers (section 8).

How to behave during the defense

- Start with the 30-second pitch, then demo. That way you steer the conversation.
- If you do not know an answer, say what you do know and name the limit honestly: “I did not measure that, but I would expect…”.
- If the teacher asks you to change something (another id range, another name prefix, another capacity range), change the value in `src\Main.ts` of that exercise, run `npm start` again and explain the new result.
- The code is deliberately minimal: if asked “why no validation / tests / comments?”, answer that the task sheets ask for exactly this and the code answers only the questions asked.

4. Work No. 4 – ORM setup: DAO, sessions, transactions

The task (Java reference): a Maven project with Hibernate and the PostgreSQL driver; `hibernate.cfg.xml`; a class `manager.DAO` with a static `ThreadLocal<Session>` and a `SessionFactory`, methods `getSession()`, `begin()` and `commit()`; a class `TestSrvs` extending DAO with `getName(id)`; `Main`; two more methods – a list of values from an ID range, and an insert wrapped in begin/commit using `executeUpdate`. The task sheet does not ask for a report; one was made anyway (7 pages).

4.1 The 30-second pitch

Say this

"The reference stack is Java and Hibernate; I used the agreed alternative, TypeScript with Drizzle ORM. The mapping is one to one: pom.xml is package.json, hibernate.cfg.xml is config.ts, the SessionFactory is a connection pool, the session lives in a static field of the DAO class, and getSession, begin and commit do what the task describes. TestSrvs extends DAO and has three methods: getName by id, getNames for an id range, and addStation, which is wrapped in begin and commit. Main prints the station with id 1, the names of stations 1 to 5, and the number of inserted rows. I checked with a separate psql connection that the inserted row is really committed."

Эталон – Java и Hibernate, я использовал согласованный альтернативный стек: TypeScript и Drizzle ORM. Соответствие прямое: конфигурация – config.ts, фабрика сессий – пул соединений, сессия хранится в статическом поле DAO. TestSrvs наследует DAO и содержит три метода: getName, getNames и addStation, обёрнутый в begin и commit.

4.2 Java to TypeScript

Reference (Java)	What I did (TypeScript)
Maven, pom.xml	npm, package.json (dependencies: drizzle-orm, pg)
Hibernate	Drizzle ORM (a query builder with typed table definitions)
JDBC driver for PostgreSQL	pg (node-postgres)
hibernate.cfg.xml	`src/config.ts` (host, port, database, user; defaults point at the project's own server)
entity class with annotations	`pgTable(...)` in `src/schema.ts`
SessionFactory	`Pool` in the static field `DAO.sessionFactory`
ThreadLocal<Session>	static field `DAO.session` (JavaScript runs in one thread)
beginTransaction() / getTransaction().commit()	`BEGIN` / `COMMIT` on the session's connection
createQuery(...).list(); executeUpdate	`session.select()…`; `session.insert(…)` and `rowCount`

4.3 Demo script

1. Start the server (`npm run pg:start`), run `npm start` in pz4. Expected output: 'Площадь Революции', a list of five names, and `1` (one row inserted). Each run adds one station.
2. Open `src\DAO.ts`: point at `sessionFactory` (the pool), `session`, `getSession()` (create on first use), `begin()`, `commit()`, `close()`.
3. Open `src\TestSrvs.ts`: `getName`, `getNames` (BETWEEN, ordered by id) and `addStation` (begin, insert, commit).
4. Prove the commit: in psql run `SELECT id, name FROM station WHERE id > 300;` – the new row is visible to another connection. (Clean up afterwards: `DELETE FROM station WHERE id > 300;`.)

4.4 What you must be able to explain

- What an ORM does and why (object–table mapping, typed queries, less hand-written SQL, parameters sent separately from the SQL).
- SessionFactory versus Session: the factory (here the pool) is created once and gives out sessions (here connections); a session is one unit of work.
- Why begin and commit are needed for the insert: the row becomes permanent and visible to others only after COMMIT.
- How the DAO checks for an open session in `getSession()`, and why `close()` is needed at the end of Main.

4.5 Questions and answers

What is an ORM and why is it used?
Что такое ORM и зачем он нужен?

Object-Relational Mapping connects program objects and database tables, so you write queries through typed code instead of strings. Benefits: fewer mistakes, parameters sent separately (no SQL injection), the same code for different databases. Price: another layer to learn and sometimes less control over the SQL.

Say it in Russian: «ORM связывает объекты программы и таблицы базы данных: запросы строятся типизированным кодом, а не строками. Меньше ошибок и защита от SQL-инъекций.»

Why Drizzle and not Hibernate?
Почему Drizzle, а не Hibernate?

Hibernate is a Java library. The agreed stack for the course work is Node.js and TypeScript, where Drizzle is a typed ORM for PostgreSQL; every Hibernate element of the task has a counterpart (table in the report).

What is the SessionFactory and what is the Session here?
Что такое SessionFactory и Session в вашей реализации?

`DAO.sessionFactory` is a `Pool` of database connections created once from `config.ts`. A session is a Drizzle object built on one connection taken from the pool; `getSession()` creates it on first use and keeps it in `DAO.session`.

What replaces ThreadLocal?
Что заменяет ThreadLocal?

In Java every thread needs its own session, hence ThreadLocal. JavaScript code runs in a single thread, so a static field is enough for this exercise. In work 6, where several requests are served at the same time, the session is kept per request with `AsyncLocalStorage`, which is the real analogue of ThreadLocal.

What do begin() and commit() do? What if you forget commit()?
Что делают begin() и commit()? Что будет, если забыть commit()?

`begin()` sends BEGIN, `commit()` sends COMMIT on the session's connection. Until COMMIT other connections do not see the change, and if the connection is closed without it, the database discards the change. That is why the insert method is wrapped in begin/commit, while the read methods need no explicit transaction.

Is the query built by string concatenation? What about SQL injection?
Запрос собирается конкатенацией строк? Что с SQL-инъекциями?

No. Drizzle sends the SQL text with placeholders and the values separately. I checked it: for `where id = '1; DROP TABLE station'` the generated SQL is `select "name" from "station" where "station"."id" = $1` and the text travels as the parameter, so it is never executed as SQL.

Why does Main call DAO.close()?
Зачем Main вызывает DAO.close()?

The session holds a connection taken from the pool, and the pool holds open sockets; while they exist the Node.js process does not exit. `close()` releases the connection and ends the pool.

What does getNames(from, to) return exactly?
Что именно возвращает getNames(from, to)?

The names of stations whose id is between `from` and `to` inclusive (BETWEEN), ordered by id; for 1 and 5 it is five names: Площадь Революции, Городской сад, Морской вокзал, Приморский бульвар, Заречье.

What is the result of addStation and how can you check it?
Что возвращает addStation и как это проверить?

The number of inserted rows (`rowCount`), that is 1. To check, read the table from another connection, for example in psql: `SELECT id, name FROM station WHERE id > 300`.

5. Work No. 5 – entities, Criteria API, repositories

The task (Java reference): Entity classes for all tables; a DAO with `findAll` through the Criteria API; the work-4 methods rewritten with Criteria and Restrictions; a method with dynamic filters; get, save, delete and a one-to-many demo; Spring Data repositories; a query method and one method with `@Query`. The report must show Entity/DAO/Repository classes, call examples and console output (11 pages).

5.1 The 30-second pitch

Say this

"All nine tables are mapped in model.ts with their columns, keys and foreign keys, and the one-to-many relation district–station is declared in relations.ts on both sides. Criteria is a small class on top of the query builder that collects conditions and ordering, with a Restrictions object for eq, ge, le, between and like. StationDao uses it for findAll, getName, listByIdRange, a prefix search and the dynamic search method: a condition is added only when its parameter is not null. The one-to-many demo creates two stations of a district in a transaction, shows the count 33 → 35 and removes them again. For the Spring Data part there is a generic JpaRepository with findAll, findById, save and deleteById, a StationRepository with a query method and a method with a JOIN, and a DistrictRepository."

Все девять таблиц описаны в model.ts; связь район–станция один ко многим описана с двух сторон в relations.ts. Criteria накапливает условия и сортировку. В методе search условие добавляется только если параметр задан. Репозитории: общий JpaRepository, StationRepository с методом по имени и методом с JOIN, DistrictRepository.

5.2 Java to TypeScript

Reference (Java)	What I did (TypeScript)
@Entity, @Table, @Id, @GeneratedValue, @Column	`pgTable('name', { … })` in `src/model.ts` – 9 tables, `generatedByDefaultAsIdentity()`, `notNull()`, `references()`
@OneToMany(mappedBy) / @ManyToOne	`relations()` in `src/relations.ts` with `many` and `one`
session.createCriteria, Restrictions	class `Criteria` and object `Restrictions` in `src/Criteria.ts` (a thin layer over Drizzle's query builder)
AuthorDao-style DAO	`StationDao`, `DistrictDao`
JpaRepository<T, ID>	generic class `JpaRepository`: findAll, findById, save, deleteById
query method findByNameStartingWithIgnoreCase	same method name on `StationRepository`, written by hand with `ilike`
@Query with JPQL join	`findStationsByDistrictName`: an explicit JOIN of station and district

5.3 Demo script

1. `npm start` in pz5 (server running). The output has 15 lines; the key ones: `findAll: 300`; `search("Пл", null, null): 1`; `search(null, 16, 18): 29`; `district Центральный: stations 33 -> 35`, `via relation: 35`, `after delete: 33`; `repository findAll: 300`; `findStationsByDistrictName("Центральный"): 33`.
2. Open `src\model.ts` (first lines: district and station) and `src\relations.ts`.
3. Open `src\Criteria.ts`: `add`, `addOrder`, `list`, `uniqueResult`; then `StationDao.search`: three `if (… !== null)` lines add conditions.
4. Open `src\JpaRepository.ts` and `StationRepository.ts`.

5.4 What you must be able to explain

- How a table is mapped to an entity, and what the foreign key and the relation declaration are for.
- What Criteria and Restrictions are, how a dynamic filter works, and why it is better than concatenating SQL strings.
- What the Repository pattern is and what Spring Data does automatically. **Be honest:** Spring Data derives the query from the method name and generates the implementation; in Node.js there is no such framework, so `findByNameStartingWithIgnoreCase` is written by hand with the same name and the same behaviour.
- The one-to-many demo and fetching: Drizzle loads related rows only when you ask (`with: { stations: true }`); nothing is loaded lazily behind your back.

5.5 Questions and answers

What is an Entity and how is it mapped?
Что такое Entity и как оно отображается?

An entity is a class (in Java) that represents a table row. Here the mapping is a table definition: `pgTable('station', { id, districtId, name, … })` lists the columns, their types and constraints; the types of a row for reading and for inserting are derived from it (`$inferSelect`, `$inferInsert`).

Say it in Russian: «Entity – класс, представляющий строку таблицы. Здесь отображение задаётся определением таблицы: столбцы, типы и ограничения.»

How is the one-to-many relation district–station declared?
Как описана связь «один ко многим» район–станция?

Twice: the table definition of station has a foreign key to district, and `relations.ts` says that a district has `many` stations and a station has `one` district (through `districtId`). Then `getWithStations` returns a district together with its stations.

What is the Criteria API and how is it different from SQL?
Что такое Criteria API и чем оно отличается от SQL?

It builds a query from objects and method calls instead of a text: `criteria.add(Restrictions.eq(station.id, 1))`. The query is checked by the compiler and is built piece by piece, which is ideal for dynamic filters. The same result can be written as SQL, but conditions would have to be glued together as strings.

How does the dynamic filter search() work?
Как работает динамический фильтр search()?

Every parameter may be null. For each non-null parameter one condition is added (`ilike` for the name, `ge` for the lower and `le` for the upper capacity); with no parameters there is no WHERE at all. Measured: name ‘Пл’ → 1 station; capacity 16–18 → 29 stations; name ‘Пл’ and capacity ≥ 10 → 1.

Which Restrictions did you implement?
Какие Restrictions вы реализовали?

`eq`, `ge`, `le`, `between`, `like` and `ilike` – thin names over the Drizzle operators; `Order.asc` and `Order.desc` for sorting.

How did you rewrite the work-4 methods?
Как вы переписали методы четвёртой работы?

`getName(id)` is `Restrictions.eq(station.id, id)` with `uniqueResult()`; `listByIdRange(from, to)` is `Restrictions.between` with ordering by id; the results are the same as in work 4.

What is the Repository pattern and what does JpaRepository give?
Что такое паттерн Repository и что даёт JpaRepository?

A repository hides data access behind a collection-like interface. A generic `JpaRepository` gives any entity findAll, findById, save and deleteById without writing them again; concrete repositories extend it and add their own queries. In Spring Data the framework generates the implementation from an interface; here it is a generic class.

What is a query method? What is @Query?
Что такое query method? Что такое @Query?

A query method is a method whose name is the query: in Spring Data `findByNameStartingWithIgnoreCase(prefix)` becomes ‘name starts with prefix, ignoring case’. `@Query` holds an explicit JPQL or SQL text for what a name cannot express, for example a join. My analogue of the second is `findStationsByDistrictName`, which joins station and district and filters by the district name (33 stations for ‘Центральный’).

How is the one-to-many demo performed?
Как выполнена демонстрация связи один ко многим?

Inside begin/commit two stations are saved for district 1; the count of the district’s stations goes from 33 to 35 (also visible through `getWithStations`); then both are deleted in a second transaction and the count returns to 33.

What is lazy and eager fetching?
Что такое lazy и eager загрузка?

Lazy loads related rows only when they are first used; eager loads them together with the parent. Hibernate has both. Drizzle has no hidden lazy loading: you request the relation explicitly (`with: { stations: true }`), so it is always explicit and eager; this avoids the classic N+1 surprise.

6. Work No. 6 – REST service and transactions

The task (Java reference): a Spring Boot REST service with GET (list), POST (create) and DELETE (by id); JSON; a DTO or projection from several tables; a service layer with `@Transactional` for a composite operation; a rollback demonstration; an optional demo client. The report must show the endpoint URLs, request examples, the transactional service code, a rollback screenshot and the default PostgreSQL isolation level (13 pages).

6.1 The 30-second pitch

Say this

"The service is built with Express on Node.js; the layers are the same as in Spring: routes in app.ts are the controllers, services.ts is the service layer, and Drizzle with the DAO from the earlier works is the data layer. It has GET, POST and DELETE for stations, GET and DELETE and POST for clients, and GET /api/rentals which returns a DTO built from four tables. A missing record gives 404 through an error handler. The transaction is DAO.transactional, my analogue of @Transactional with propagation REQUIRED: begin, run, commit, and rollback on any exception; read methods use a READ ONLY transaction. The composite operation registers a client, a wallet and favourite stations in one transaction; with ?fail=true the service throws after saving, and I showed with psql that none of the three rows exist. Without the error all three are saved. PostgreSQL's default isolation level is READ COMMITTED. The demo client is an HTML page with fetch."

Сервис на Express: маршруты – контроллеры, services.ts – сервисный слой, Drizzle – доступ к данным. GET /api/rentals возвращает DTO из четырёх таблиц. Транзакция – DAO.transactional (аналог @Transactional): фиксация при успехе, откат при исключении. С параметром fail=true после сохранения возникает ошибка, и в базе нет ни клиента, ни кошелька, ни избранных станций. Уровень изоляции по умолчанию – READ COMMITTED.

6.2 Java to TypeScript

Reference (Java)	What I did (TypeScript)
Spring Boot, starter-web	Express 5
@SpringBootApplication, SpringApplication.run; 'Started Application on port 8080'	`src/Application.ts`: `app.listen(8080)` and the same message
@RestController, @GetMapping, @PostMapping, @DeleteMapping	`app.get`, `app.post`, `app.delete` in `src/app.ts`
@RequestBody; ResponseEntity.ok / noContent	`express.json()`; `res.json()`, `res.status(204).end()`
DTO record	interface `RentalDto`
@Service, @Transactional, @Transactional(readOnly = true)	classes in `src/services.ts`; `DAO.transactional(fn, { readOnly })`
@ExceptionHandler(EntityNotFoundException) → 404	error-handling middleware: `NotFoundError` → 404
ThreadLocal session per request	`AsyncLocalStorage` in `DAO.run`, started by the `route()` wrapper for every request

6.3 The endpoints

Method	URL	What it does	Answer
GET	/api/stations	list of stations (300)	200
GET	/api/stations/{id}	one station	200 / 404

Method	URL	What it does	Answer
POST	/api/stations	create a station from the JSON body	200
DELETE	/api/stations/{id}	delete a station	204 / 404
GET	/api/rentals?limit=N	DTO from rental + client + station + tariff, newest first	200
GET	/api/clients/{id}	one client	200 / 404
POST	/api/clients[?fail=true]	register a client + wallet + favourite stations in one transaction; `fail=true` throws after saving	200 / 500
DELETE	/api/clients/{id}	delete a client (wallet and favourites go with it)	204 / 404
GET	/	the demo client (public/index.html)	200

POST returns 200 because the task sheet says so (“верните созданный объект с 200 OK”); 201 Created would be the stricter REST choice.

6.4 Demo script

1. Start the database (`npm run pg:start`), then `npm start` in pz6: ‘Started Application on port 8080’. In another terminal or in the browser: `curl http://localhost:8080/api/stations/1`, `curl "http://localhost:8080/api/rentals?limit=2"`, `curl http://localhost:8080/api/stations/999999` (404 with `{"error":"station 999999 not found"}`).
2. POST and DELETE: write the JSON into a file (`station.json`) because Russian text typed straight into a PowerShell or Git Bash command line can arrive as ‘?’: `curl -X POST http://localhost:8080/api/stations -H "Content-Type: application/json" -d @station.json` then `curl -X DELETE http://localhost:8080/api/stations/<id>` (204).
3. The rollback: the same POST to `/api/clients?fail=true` returns 500 `{"error":"test rollback"}`; then in psql count the client, its wallet and favourites (queries in report 6, figures 5 and 6): **0, 0, 0**. Without `fail=true` the answer is 200 and the counts are **1, 1, 2**. Delete the test client afterwards.
4. Or just run `npm run demo`: it starts the service on a free port, runs the whole scenario, checks the database, deletes what it created and prints each step.
5. Open `http://localhost:8080/` in a browser: the demo client shows the stations in a table. Open `src\DAO.ts` (method `transactional`) and `src\services.ts` (`ClientService.register`).

6.5 What you must be able to explain

- REST basics: resources and URLs, the meaning of GET, POST and DELETE, status codes 200, 204, 404, 500, JSON.
- The layers: controller (routes) → service → data access; why business logic and transactions belong in the service.
- A transaction: atomicity (all or nothing); commit and rollback; how `DAO.transactional` works; propagation REQUIRED (a nested call joins the open transaction); read-only transactions.
- The default isolation level of PostgreSQL: READ COMMITTED; what it allows and forbids.
- The demo: what was in the transaction, where the exception is thrown, and what the database contained afterwards.

6.6 Questions and answers

What is REST?
Что такое REST?

An architecture style for web services: resources are addressed by URLs and manipulated with the HTTP methods – GET reads, POST creates, PUT/PATCH changes, DELETE removes – and answered with status codes and usually JSON. The service is stateless: every request carries what is needed.

Say it in Russian: «REST – стиль веб-сервисов: ресурсы адресуются URL, операции – методы HTTP, ответы – коды состояния и JSON. Сервис не хранит состояние между запросами.»

Which status codes do you return and when?
Какие коды ответа вы возвращаете и когда?

200 for a successful GET and POST (as the task sheet says; 201 would be stricter), 204 with an empty body for a successful DELETE, 404 when the record does not exist (from `NotFoundError`), 500 for an unexpected error.

What is a DTO and why do you need it?
Что такое DTO и зачем он нужен?

A Data Transfer Object is a shape made for the answer, not a table row. `RentalDto` combines the rental id, the client's full name, the start station, the tariff code, the start time and the cost from four tables, so the client gets one flat JSON object and the internal table structure stays hidden.

How does the service layer differ from the controller?
Чем сервисный слой отличается от контроллера?

The controller (route) only receives the request and sends the answer; the service holds business logic and transaction boundaries and is independent of HTTP. That way the same service can be called from a route, from the demo script or from a test.

What does @Transactional do, and what is your analogue?
Что делает @Transactional и что у вас ему соответствует?

It wraps a method in a transaction: if the method returns normally the transaction is committed, if it throws a RuntimeException it is rolled back. My analogue is `DAO.transactional(fn, options)`: it runs BEGIN, calls `fn`, runs COMMIT, and on any exception runs ROLLBACK and rethrows. If a transaction is already open it just calls `fn` (propagation REQUIRED).

Say it in Russian: «@Transactional оборачивает метод в транзакцию: при успехе фиксация, при исключении откат. У меня это DAO.transactional: BEGIN, выполнение, COMMIT, а при ошибке ROLLBACK.»

What is propagation REQUIRED and where do you see it?
Что такое propagation REQUIRED и где это видно?

REQUIRED means: join the current transaction if there is one, otherwise start a new one. In `DAO.transactional` the first line checks the scope's `inTransaction` flag and, if it is set, simply returns `fn()`; so a service method called from another transactional method does not open a second transaction.

What does readOnly = true do?
Что делает readOnly = true?

It opens the transaction with `BEGIN READ ONLY`: PostgreSQL then refuses any write in it, which protects read methods from accidental changes and lets the database optimise. All GET methods of my services use it.

Show the rollback. What exactly was rolled back?
Покажите откат. Что именно было отменено?

`ClientService.register` saves a client, its wallet and its favourite stations in one transaction and, with `fail=true`, throws `Error('test rollback')` after saving. `DAO.transactional` catches it, sends ROLLBACK and rethrows; the route's error handler answers 500. Afterwards psql shows 0 clients, 0 wallets, 0 favourites for that phone. Without the flag the answer is 200 and psql shows 1, 1, 2.

What is the default isolation level in PostgreSQL and what does it mean?
Какой уровень изоляции по умолчанию в PostgreSQL и что он означает?

READ COMMITTED (I checked with `SHOW default_transaction_isolation`). Every statement sees only data committed before that statement started. There are no dirty reads, but two identical queries in one transaction may give different results if another transaction commits in between (non-repeatable read). Stricter levels are REPEATABLE READ and SERIALIZABLE.

Say it in Russian: «READ COMMITTED: каждая команда видит только зафиксированные данные. Грязного чтения нет, но возможно неповторяющееся чтение.»

How do you keep one database session per request?
Как у вас обеспечивается одна сессия на запрос?

The `route()` wrapper runs every handler inside `DAO.run`, which creates a scope with `AsyncLocalStorage` (the Node.js analogue of ThreadLocal) and releases the connection at the end. Concurrent requests therefore never share a connection or a transaction.

How is the demo client built?
Как устроен демо-клиент?

`public/index.html` is a static page served by `express.static`: it calls `fetch('/api/stations')`, takes the JSON and fills an HTML table.

What is not in your service? What would you add?
Чего нет в вашем сервисе? Что бы вы добавили?

The code is minimal on purpose, so: **no input validation** – an invalid body or a non-numeric id ends as a database or parsing error and a **500** answer, and I checked that the error text then contains the SQL statement, which a production service should not expose; no authentication; no pagination (GET /api/stations returns all 300 rows); POST returns 200 instead of 201. I would add validation with 400 answers, error messages without SQL, 201 Created and pagination.

Why is POST /api/clients?fail=true a safe way to demonstrate rollback?
Почему параметр fail=true удобен для демонстрации отката?

The error is thrown after all three inserts have run, so it proves that already executed statements are undone; and it is part of the demo only: without the parameter the service behaves normally. In the Java sheet the same is shown by a commented-out `throw new RuntimeException`.

7. If something goes wrong

Problem	What to do
psql says “connection refused”, or `npm start` cannot connect	The server is not running: `npm run pg:start` in any of the folders of works 2–6. The data is the folder `.pgdata`, port 54329.
`pg:start` seems to hang	After a crash or a power-off PostgreSQL recovers first; this can take a minute or more. Wait for “запущен”.
Port 54329 is busy	Another PostgreSQL server of this project is running (the original project’s copy). Stop it first, then start this one.
Russian letters in psql look like ???? or garbage	`$env:PGCLIENTENCODING = 'UTF8'` before psql, and use Windows Terminal. For curl, put Russian JSON into a file and use `-d @file.json`.
Work 5 prints 301 stations (and 34 → 36 → 34 for the district), or GET /api/stations returns 301 rows	Work 4 was run and left its station. In psql: `DELETE FROM station WHERE id > 300;`, then run the exercise again.
Work 4 keeps adding stations	By design: every `npm start` inserts one. Remove them with `DELETE FROM station WHERE id > 300;` in psql.
Only 60 stations, the tables have only about 28,000 records	Someone ran `npm start` in pz2. Run `npm start` in pz3: it loads the full data again (about a minute); then 300 stations are back.
The database is damaged or empty	`npm start` in pz2, then `npm start` in pz3: the data is reproducible (fixed seeds), the row counts will match the reports.
Port 8080 is busy (work 6)	Another instance of the service is running. Close it, or find it with `netstat -ano | findstr :8080` and stop that process.
Why not the main PostgreSQL on port 5432?	The project has its own cluster in its folder: no password of the main server needed, other databases untouched, fully reproducible.

8. Cheat sheet: numbers and rules

Key numbers

Work	What	Value
–	State of the database before the defense	9 tables; 10 districts, 300 stations, 50,000 clients, 400,000 rentals; PostgreSQL 17 on port 54329, database `velograd`
4	Result of npm start	‘Площадь Революции’; five names (ids 1–5): Площадь Революции, Городской сад, Морской вокзал, Приморский бульвар, Заречье; 1 row inserted (one more station on every run)
5	Results	findAll 300; search 1 / 29 / 1; district Центральный 33 → 35 → 33 stations; 10 districts; findStationsByDistrictName 33
6	Service	port 8080; 9 routes; 200 / 204 / 404 / 500; isolation READ COMMITTED; rollback: 0, 0, 0 rows; success: 1, 1, 2 rows

Key rules

- ON DELETE is a rule of the database: RESTRICT for reference data and history, CASCADE for dependent rows (deleting a client removes its wallet and favourite stations), SET NULL for soft links.
- A transaction is all or nothing: COMMIT on success, ROLLBACK on error. Default isolation in PostgreSQL: READ COMMITTED.
- Never glue user input into SQL: an ORM sends values as parameters ($1) separately from the SQL text.
- REST: GET reads (200), POST creates (200 here, 201 is stricter), DELETE removes (204), a missing record is 404.

9. Glossary English – Russian

English	Русский	English	Русский
primary key	первичный ключ	dynamic filter	динамический фильтр
foreign key	внешний ключ	lazy / eager loading	ленивая / жадная загрузка
constraint	ограничение	transaction	транзакция
one-to-many relation	связь один ко многим	commit / rollback	фиксация / откат
cascading delete	каскадное удаление	isolation level	уровень изоляции
join	соединение (JOIN)	propagation	распространение транзакции
ORM (object-relational mapping)	объектно-реляционное отображение	read-only transaction	транзакция только для чтения
DAO (data access object)	объект доступа к данным	REST endpoint	конечная точка REST
session factory / session	фабрика сессий / сессия	DTO (data transfer object)	объект передачи данных
connection pool	пул соединений	status code	код состояния HTTP
entity	сущность	request body	тело запроса
mapping	отображение (маппинг)	service layer	сервисный слой
Criteria API	программный интерфейс построения запросов	controller	контроллер
repository	репозиторий	middleware	промежуточный обработчик
query method	метод запроса по имени		